Lesson 07 · Solutions, MSBuild & Mono-repos

One solution, many projects — MSBuild evaluation, Directory.Build.* inheritance, central package management, a build graph whose cost you can predict, and one CI workflow for the repo

▤ Contents

1 · Why a .NET mono-repo looks empty 1
- 1.1 · The numbers to leave this lesson with 2

2 · Solutions — a list of projects, not a build script 2
- 2.1 · The same five projects, two solution formats 3
- 2.2 · The whole solution file 3
- 2.3 · Concept map — Gradle, Maven and Lerna to MSBuild 3
- 2.4 · A solution filter lists three projects and builds four 4

3 · The MSBuild model — two phases, four nouns 5
- 3.1 · What happens before your code compiles 5
- 3.2 · The nouns of a build, and what you call them today 5
- 3.3 · Where the lines of an evaluated project come from 6
- 3.4 · Items and targets — the repository's own build logic 6
- 3.5 · Running one target and nothing else 6

4 · Shared build logic — and the one rule that bites 7
- 4.1 · Who imports what, measured on L07.Pricing 7
- 4.2 · The nested Directory.Build.props, line by line 7
- 4.3 · What the trap actually looks like 8
- 4.4 · The parent it chains to — and how one file serves C# and VB 8
- 4.5 · The Directory.* family — what each file may decide 9

5 · Dependencies — one version list for the repository 9
- 5.1 · The repository's one version list 10
- 5.2 · The consumer side — references without versions 10
- 5.3 · Four ways a shared version list fails, and what restore says 10
- 5.4 · What each ecosystem's shared version list actually guarantees 11

6 · The build graph — what recompiles when you change one line 11
- 6.1 · The MotorMono build graph, and what each edge carries 12
- 6.2 · Projects the compiler re-ran, by kind of edit 12
- 6.3 · internal in Kotlin and in C# — the same word, a different boundary 12
- 6.4 · One build graph, two languages, one interface 13

7 · Quality gates that live inside the build 13
- 7.1 · The whole style configuration 13
- 7.2 · How many code-analysis rules each analysis mode turns on 14
- 7.3 · Every gate in this repository, and what you knew it as 14

8 · Packaging and CI — one workflow for the whole repository 14
- 8.1 · The build half of the workflow 15
- 8.2 · One push through the pipeline 16
- 8.3 · The publish half, and the secret that is not a secret 16
- 8.4 · The packaging decisions, and what MotorMono chose 16

9 · Hands-on — drive MotorMono, then take it apart 17
- 9.1 · Commands 17
- 9.2 · What you should see 17
- 9.3 · Lines of code per module 18
- 9.4 · What carries over from Gradle, what to learn, what to unlearn 18

Page numbers are resolved from the rendered PDF. Code panels are cut from the compiled samples; every build figure is measured by `samples/tools/measure_build.py` at build time.

1 · Why a .NET mono-repo looks empty

Open a .NET mono-repo and the project files look like someone forgot to finish them. `L07.Pricing.csproj` in this lesson's samples is 5 lines long and names no target framework, no source files, no compiler settings and no package versions. That is not a minimal example: it is what a production project file looks like. The build did not disappear — it moved into two

places you cannot see from the project file, the SDK and a small family of files named `Directory.*` that sit above it in the folder tree.

Those 5 lines evaluate to 17,412 lines of MSBuild. `dotnet msbuild -pp` expands every import into one document and marks each one with a banner, 1,625 lines in all. Of the other 15,787 lines, 15,696 come from the .NET SDK, 74 from the repository's own `Directory.*` files, 12 from restore output and 5 from the project file itself. Coming from a Kotlin mono-repo you are used to reading a module's `build.gradle.kts` to know how it builds. Here you read upwards, and the first question about any .NET repository is *which files above this one are changing the build*.

The vocabulary maps cleanly; the inheritance rule does not. Properties are Gradle's `ext` values, items are source sets, targets are tasks, the SDK is a convention plugin. But where Gradle merges a root script with every subproject script, MSBuild imports the **nearest** `Directory.Build.props` and then stops looking — a nested one must import its parent explicitly or the parent silently stops applying. That single rule is the most expensive thing in this lesson, and §4 shows what it costs when you miss it.

What you get in exchange is a build graph you can reason about numerically. Because the compiler writes a reference assembly next to each output, MSBuild can tell whether a change to a project can possibly affect the projects that depend on it. Change a method body in this repo and one project recompiles; change a public signature and all 5 do. §6 measures every case, including the one where an *internal* change behaves like a public one.

1.1 · The numbers to leave this lesson with

MOTORMONO	A 5-LINE PROJECT FILE	SOLUTION FILE	ONE PUBLIC API CHANGE	ANALYZER RULES ON
5 projects	**17,412 lines**	**16 lines**	**5 of 5 recompile**	**145 CA rules**
C# + VB + tests, one solution · measured	after evaluation · dotnet msbuild -pp	the same 5 projects as .sln: 101 · measured	an internal one in L07.Core: 1 · measured	latest-recommended, as warnings · measured

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured tooling

ℹ Before you start

1. **You need lesson 01 (The .NET Platform Map) for target framework monikers, the CLI and the publish options, and enough C# to read a class from lesson 02 (C# Language Essentials) and lesson 03 (Types, OOP & Generics).** The Visual Basic project in this lesson only has to be readable; lesson 06 (VB.NET for Legacy Estates) teaches the language and the migration plan.
2. **You will build `samples/MotorMono`, a mini mono-repo inside this repository.** 5 projects in `src/` and `tests/`, one solution file, a nested `Directory.Build.props` that imports the repository's, central package management, a custom target, a solution filter and a GitHub Actions workflow that is read into this PDF but never runs here.
3. **Run it with** `python 0-script/verify_samples.py --only 7` from the repository root. It must report 5/5 projects passed; the test project alone contributes 14 tests.
4. **Every build number in this lesson is measured, not quoted.** `samples/tools/measure_build.py` copies MotorMono to a temporary folder, edits it, builds it, breaks it on purpose and records what happened in `samples/tools/measurements.json` — re-measured on 2026-09-20 with SDK 10.0.401 and MSBuild 18.9.11.42413 on Windows.
5. Facts that change with releases are marked • Verified and link to the official source; numbers computed from this repository are • Measured; a judgement is • Estimate. Gradle, Kotlin and TypeScript panels are for comparison and are not compiled.

2 · Solutions — a list of projects, not a build script

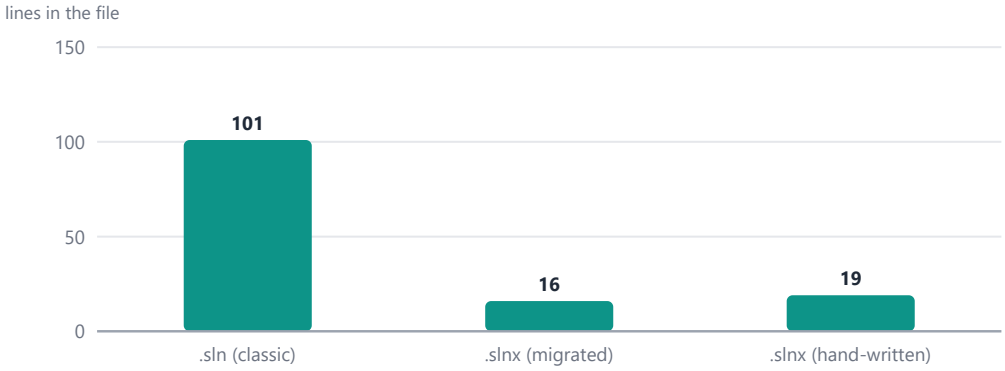
A solution file is an index, and that is all it is. It names the projects an IDE should load and a command-line build should walk. It defines no tasks, is not imported by any project and carries no shared build settings beyond a per-configuration mapping. Its only say in build order is the solution-level project dependencies an IDE can add, which MSBuild turns into `ProjectReference` items — the order the projects themselves declare is what a mono-repo runs on. If you are looking for the mono-repo's shared configuration, the solution file is the one place it is guaranteed not to be.

In .NET 10 the format is XML, and it is the default. `dotnet new sln` now produces `.slnx`; the .NET SDK gained the format in 9.0.200 and `--format sln` still writes the old one (dotnet new sln defaults to SLNX · SLNX support in the .NET CLI). The old `.sln` is a line-oriented format from Visual Studio .NET whose content is mostly GUIDs and per-configuration mappings; the

new one is a project list. Visual Studio reads both from 17.14 (the simpler solution file format), and MSBuild has accepted `.slnx` since 17.12 (How MSBuild builds projects).

What you actually gain is merge behaviour. The measured comparison below is the same 5 projects written both ways: 101 lines and 10 distinct GUIDs against 16 lines and none. Two people adding a project to a `.sln` on two branches conflict in a block of generated identifiers; on a `.slnx` they add one line each. This is the same reason your Gradle mono-repo keeps `settings.gradle.kts` boring.

2.1 · The same five projects, two solution formats



measured by creating the classic file with the CLI, adding the five projects, then running dotnet sln migrate on the build machine

101 lines and 10 GUIDs become 16 lines and none. The classic file is 6,888 characters against 530 — most of it per-project, per-configuration mappings that merge badly. The hand-written file in this repository is 19 lines because it adds solution folders for the build files. • Measured, and • Verified as the .NET 10 default (dotnet new sln defaults to SLNX).

2.2 · The whole solution file

MSBuild / XML — `samples/MotorMono/MotorMono.slnx`

```
<Solution>
  <Folder Name="/build/">
    <File Path="Directory.Build.props" />
    <File Path="Directory.Build.targets" />
    <File Path="Directory.Build.rsp" />
    <File Path="Directory.Packages.props" />
    <File Path=".editorconfig" />
    <File Path="nuget.config" />
  </Folder>
  <Folder Name="/src/">
    <Project Path="src/L07.Core/L07.Core.csproj" />
    <Project Path="src/L07.Pricing/L07.Pricing.csproj" />
    <Project Path="src/L07.Pricing.Vb/L07.Pricing.Vb.vbproj" />
    <Project Path="src/L07.QuoteCli/L07.QuoteCli.csproj" />
  </Folder>
  <Folder Name="/tests/">
    <Project Path="tests/L07.Pricing.Tests/L07.Pricing.Tests.csproj" />
  </Folder>
</Solution>
```

Folders, projects, nothing else. The `/build/` folder is a *solution folder* — a display grouping that makes the shared build files visible in an IDE; it creates no directory and changes no build. Note what is absent: no configuration matrix, no project GUIDs, no build order. `dotnet build MotorMono.slnx` builds these projects in dependency order because each project declares its own `ProjectReference` items (6.1), not because this file says so.

2.3 · Concept map — Gradle, Maven and Lerna to MSBuild

You already know	In .NET	Kind	Watch for
`settings.gradle.kts` · `lerna.json` · Maven `<modules>`	`MotorMono.slnx`	• Renamed	A list of projects only: no build order, no shared settings, nothing imports it
A module's `build.gradle.kts`	`.csproj` / `.vbproj`	• Renamed	5 lines is normal — the SDK and the Directory.* files supply the rest

You already know	In .NET	Kind	Watch for
Root `subprojects { }` / `allprojects { }`	`Directory.Build.props`	• Trap	Only the NEAREST one is imported; a nested file must import its parent by hand (4.1)
A convention plugin · `buildSrc`	`Directory.Build.targets` or a custom SDK	• Different	Imported after the project file, so it can override what a project set
`gradle.properties`	Properties in `Directory.Build.props`	• Same idea	Plain name/value pairs, evaluated in order of appearance
`-PsomeFlag=true`	`-p:SomeFlag=true`	• Same idea	A global property: it beats any value a project file sets, though the project is read later
A Gradle task · a task action	An MSBuild target, which calls tasks (`Csc`, `Message`)	• Renamed	Ordered by `DependsOnTargets` / `BeforeTargets` / `AfterTargets`
`sourceSets` / include patterns	Items (`Compile`, `ProjectReference`)	• Renamed	The SDK already globs every `.cs`: adding one by hand fails the build with `NETSDK1022` unless `EnableDefaultCompileItems` is off
`gradle/libs.versions.toml`	`Directory.Packages.props`	• Different	A catalog offers versions; central package management plus pinning enforces them (5.4)
A Gradle `platform` · a Maven BOM	`CentralPackageTransitivePinningEnabled`	• Same idea	The switch that makes your list win for packages you never referenced
`gradle.lockfile` · `package-lock.json`	`packages.lock.json`	• Renamed	Opt-in per project via `RestorePackagesWithLockFile`, and one file per project
`npm ci` · `--offline`	`RestoreLockedMode`	• Renamed	Turn it on only when `CI` is set, or every local version bump fails
ktlint · Spotless · ESLint	.editorconfig + Roslyn analyzers + `dotnet format`	• Different	The rules ship inside the SDK; severity is per rule in `.editorconfig`
A SonarQube quality gate	`AnalysisLevel` + `TreatWarningsAsErrors`	• Different	In-build and per-project rather than a separate server pass (7.2)
Kotlin `internal` (module = source set)	`internal` + `InternalsVisibleTo`	• Trap	Gradle gives the test source set access for free; .NET needs the attribute (6.3)
`mvn install` · `npm publish`	`dotnet pack` · `dotnet nuget push`	• Renamed	One `.nupkg` per packable project, versioned from one property (8.1)
Maven Central · the npm registry	nuget.org · GitHub Packages · AWS CodeArtifact	• Same idea	Map every package id to one feed or CPM warns NU1507 (5.3)

2.4 · A solution filter lists three projects and builds four

Text `samples/MotorMono/MotorMono.Pricing.slnf`

```
{
  "solution": {
    "path": "MotorMono.slnx",
    "projects": [
      "src\\L07.Core\\L07.Core.csproj",
      "src\\L07.Pricing\\L07.Pricing.csproj",
      "tests\\L07.Pricing.Tests\\L07.Pricing.Tests.csproj"
    ]
  }
}
```

Terminal — three listed, four built `captured on the build machine; o…`

```
$ dotnet build MotorMono.Pricing.slnf
  Determining projects to restore...
  L07.Core -> ...bin/L07.Core/debug/L07.Core.dll
  L07.Pricing -> ...L07.Pricing.dll
  L07.Pricing.Vb -> ...L07.Pricing.Vb.dll
  L07.Pricing.Tests -> ...L07.Pricing.Tests.dll
Build succeeded.  0 Warning(s)  0 Error(s)
```

> **A filter narrows what is loaded, not what is built.** The filter names 3 projects and the build produced 4: `L07.Pricing.Tests` references the Visual Basic project, so MSBuild builds it too — *"It builds a project if it's specified in the filter or referenced by a project that is built"* (Solution filters in MSBuild). Treat `.slnf` as the IDE-startup-time tool it is, not a module boundary; to cut a subgraph out of a build you have to cut the `ProjectReference`. The shortened paths are the other setting worth copying: `UseArtifactsOutput` is `true` here, so every project's `bin` and `obj` land under one `artifacts/` tree — one folder to cache in CI and one to delete (artifacts output layout).

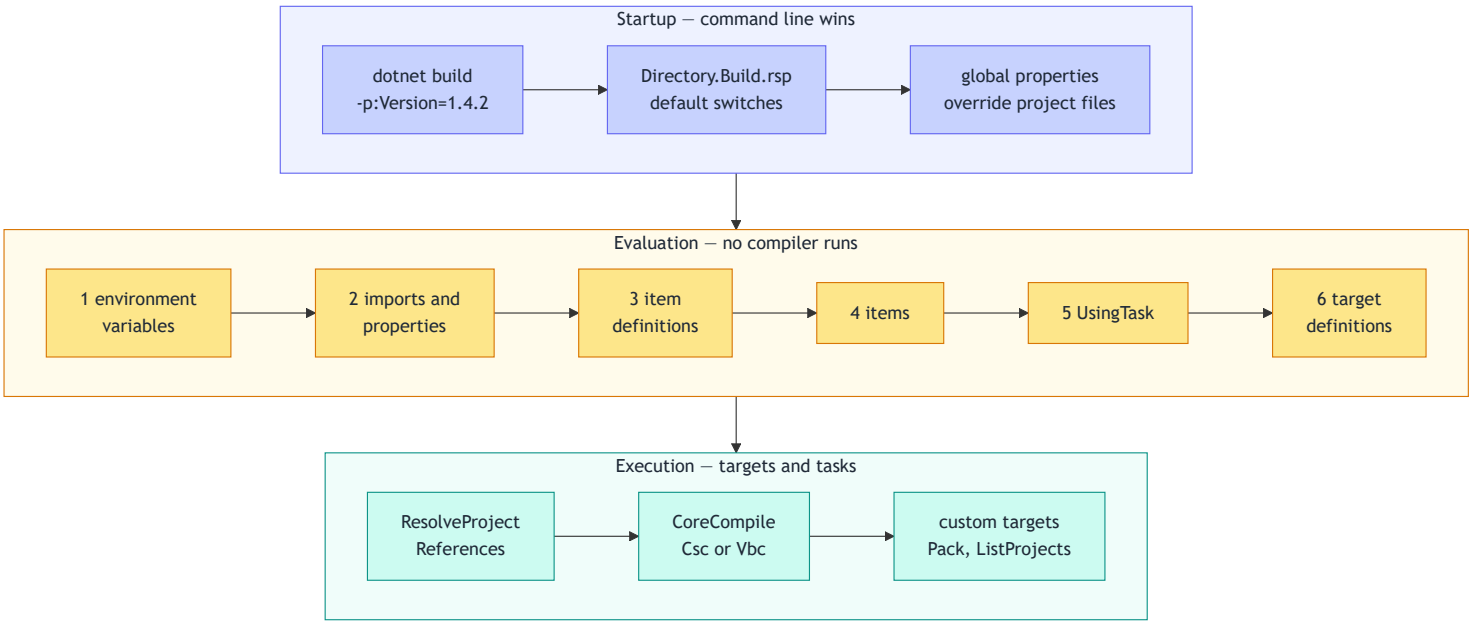
3 · The MSBuild model — two phases, four nouns

MSBuild is not a script that runs top to bottom; it is a document that is evaluated, and then a graph of targets that is executed. Everything outside a target — every property, every item, every import — is read first, in six ordered passes: environment variables, imports and properties, item definitions, items, `UsingTask` declarations, and finally target definitions. No compiler runs during any of it. Only then does execution start (How MSBuild builds projects).

Knowing which phase you are in explains the two things that surprise Gradle users most. The first: properties are evaluated in order of appearance, so a property can read values declared above it but not below — which is why `Directory.Build.props` has to be imported early to be useful and why the SDK warns you not to read items from it. The second: because the whole document is read up front, a file your build writes during execution cannot change that same build. There is no `doLast { }` that edits the build.

The nouns are few, and you already know them under other names. A property is a string. An item is a named list of things with metadata attached. A target is an ordered unit of work that calls tasks, which are .NET classes. Command-line `-p:Name=Value` creates a *global* property that overrides whatever the project file says, even though the project is read later — the same escape hatch as Gradle's `-P`. Everything else in this lesson is a particular arrangement of those four ideas.

3.1 · What happens before your code compiles



indigo = your input · amber = evaluation, where nothing is built · teal = execution, where tasks run · the six evaluation passes are ordered, and each can only see what came before it

3.2 · The nouns of a build, and what you call them today

MSBuild	What it is	Gradle · npm	In MotorMono
Property	A single string, evaluated in order of appearance outside targets	`ext` value · `gradle.properties` · a script variable	`VersionPrefix`, `AnalysisLevel`, `MotorMonoBuiltFrom`
Item	A named list; each entry has an identity and metadata, read as `%(Name)`	A `sourceSet`, a dependency configuration	`ProjectReference`, `PackageReference`, `MotorProject`, and `%(MotorProject.Filename)` in `ListProjects`
Target	An ordered unit of work with inputs, outputs and conditions	A Gradle task · an npm script	`ListProjects`, `StampBuildFacts`
Task	A .NET class the build loads and calls	A task action · an Ant task	`Message`, and `Csc` / `Vbc` from the SDK
SDK	A package of props and targets imported by the `Sdk` attribute	A convention plugin applied by id	`Microsoft.NET.Sdk` in all five projects
Binary log	A replayable recording of one build	`--scan` / a build scan	`-bl:build.binlog` in CI (the MSBuild binary log)

3.3 · Where the lines of an evaluated project come from



lines of the expanded project produced by dotnet msbuild -pp on src/L07.Pricing/L07.Pricing.csproj, attributed to the file each line came from · the 1,625 lines of import banners -pp writes between files belong to no file and are left out

You wrote 5 of 15,787 lines of build logic, and your repository contributed 74 more. The rest is the SDK: 117 files imported into one document before anything is built; the other 1,625 lines of the 17,412 are the banners `-pp` writes between files. Debugging a property means asking *where in this document is it set*, which is what `-pp` is for (Customize the build by folder). And those 74 lines from `Directory.*` are the highest-leverage lines in the repository: they apply to every module, so an error there is an error everywhere. • **Measured** .

3.4 · Items and targets — the repository's own build logic

MSBuild / XML samples/MotorMono/Directory.Build.targets

```
<Project>
  <!-- Imported LATE, after the project file, so it sees what every project has set. -->
  <ItemGroup>
    <!-- a module that HAS a test project lets it see its internal types -->
    <InternalsVisibleTo Include="$(MSBuildProjectName).Tests"
      Condition="Exists('$(MSBuildThisFileDirectory)tests/$(MSBuildProjectName).Tests')" />
    <!-- an item list built from globs: every project in the mono-repo -->
    <MotorProject Include="$(MSBuildThisFileDirectory)src/*/*.*proj" />
    <MotorProject Include="$(MSBuildThisFileDirectory)tests/*/*.*proj" />
  </ItemGroup>

  <!-- runs only when asked for:  dotnet msbuild <project> -t:ListProjects -->
  <Target Name="ListProjects">
    <Message Importance="high" Text="MotorMono: @(MotorProject->Count()) projects" />
    <Message Importance="high" Text="  %(MotorProject.Filename)%(MotorProject.Extension)" />
  </Target>

  <!-- hooked into every build of a console app: stamp build facts into the assembly -->
  <Target Name="StampBuildFacts" BeforeTargets="GetAssemblyAttributes"
          Condition="'$(OutputType)' == 'Exe'">
    <ItemGroup>
      <AssemblyMetadata Include="MotorProjects" Value="@(MotorProject->Count())" />
      <AssemblyMetadata Include="BuiltFrom" Value="$(MotorMonoBuiltFrom)" />
    </ItemGroup>
  </Target>
</Project>
```

Read this as three declarations and two behaviours. The `InternalsVisibleTo` item is conditional on a folder existing, so a module gets test access by *convention* rather than per project — the Gradle rule reproduced by hand (6.3). `MotorProject` is an item built from two globs, so `@(MotorProject->Count())` is the module count of the repository. `ListProjects` runs only when you ask for it by name; `StampBuildFacts` hooks `BeforeTargets` on an SDK target and writes those facts into every executable's metadata, which is how the CLI can print them back (9.2). Because this file is imported *after* the project file, it can safely count what the projects declared (Customize the build by folder).

3.5 · Running one target and nothing else

Terminal — a custom target, run by name captured on the build machine

```
$ dotnet msbuild src/L07.Core -t:ListProjects -nologo -tl:off

  MotorMono: 5 projects
    L07.Core.csproj
    L07.Pricing.Vb.vbproj
    L07.Pricing.csproj
    L07.QuoteCli.csproj
    L07.Pricing.Tests.csproj
```

A target you have to ask for is the safe way to add repository tooling. `-t:ListProjects` evaluates the project and runs one target, skipping the compile graph. The count and names come from the `MotorProject` globs, so a new project appears the moment its file lands on disk. The order is the file system's, not the build order.

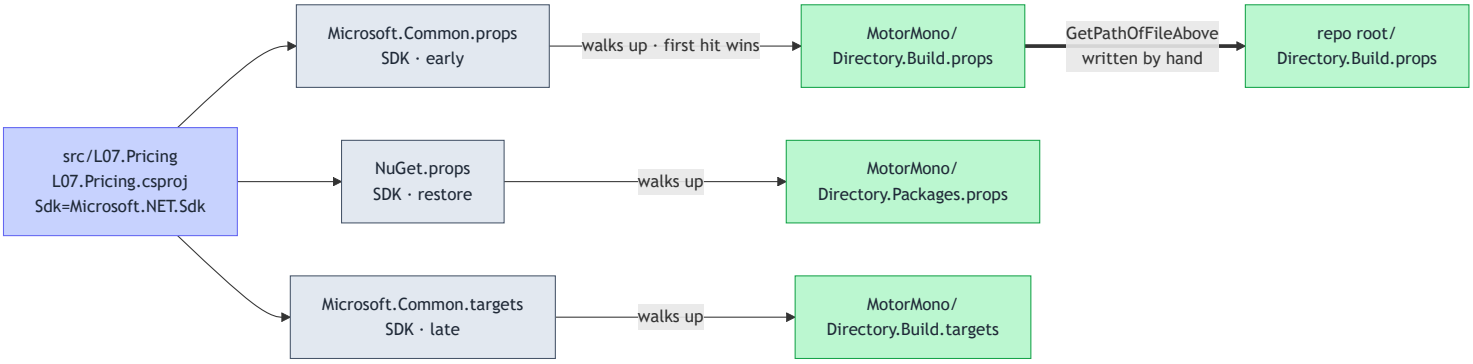
4 · Shared build logic — and the one rule that bites

MSBuild walks up from each project folder, imports the first `Directory.Build.props` it finds, and stops. Microsoft's own summary is blunt: *"the first Directory.Build.props that doesn't import anything is where MSBuild stops"* (Customize the build by folder). Gradle merges a root script into every subproject automatically; MSBuild does not: the day someone adds a second file in a subfolder, every setting from the root silently stops applying beneath it.

That is not a hypothetical — it is this lesson's samples. MotorMono has its own `Directory.Build.props` for its own version and analyzer mode, and the repository root has one that sets `net10.0`, nullable reference types, `Option Strict On` and warnings-as-errors. The nested file's first line is a hand-written `Import` of the parent (4.2, 4.4); delete it and the build fails at once with `NETSDK1013` (4.3) — the target framework has vanished.

Three other files complete the family, and the differences are about timing. `Directory.Build.props` is imported *early*, before the project file, so a project can override it — it sets defaults. `Directory.Build.targets` is imported *late*, after the project file and after any `.targets` from NuGet packages, so it wins — it enforces. `Directory.Packages.props` is pulled in by the SDK's restore logic and owns versions (§5). `Directory.Build.rsp` is read by the command-line driver only, which makes it one of the two kinds of file that can make `dotnet build` and Visual Studio disagree — the other is `Directory.Solution.props` / `.targets` (4.5).

4.1 · Who imports what, measured on L07.Pricing



indigo = the project · grey = files the SDK supplies · green = files this repository owns · every arrow was read out of the expanded project by measure_build.py; the doubled arrow is the only import written by hand

4.2 · The nested Directory.Build.props, line by line

MSBuild / XML — `samples/MotorMono/Directory.Build.props`

```
<Project>
  <!-- MSBuild imports only the NEAREST Directory.Build.props: chain to the one above by hand -->
  <Import Project="$([MSBuild]::GetPathOfFileAbove('Directory.Build.props',
                                                   '$(MSBuildThisFileDirectory)../'))" />

  <PropertyGroup>
    <!-- one version for every package this mono-repo ships; CI adds a VersionSuffix -->
    <VersionPrefix>1.4.0</VersionPrefix>
    <Company>MotorQuote sample</Company>
    <!-- the same quality gates in every module -->
    <AnalysisLevel>latest-recommended</AnalysisLevel>
    <EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>
    <!-- lock files everywhere; on CI (environment variable CI=true) they may not change -->
    <RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
    <RestoreLockedMode Condition="'$(CI)' == 'true'">true</RestoreLockedMode>
    <ContinuousIntegrationBuild Condition="'$(CI)' == 'true'">true</ContinuousIntegrationBuild>
    <!-- Directory.Build.rsp overrides this on command-line builds -->
    <MotorMonoBuiltFrom Condition="'$(MotorMonoBuiltFrom)' == ''">IDE</MotorMonoBuiltFrom>
  </PropertyGroup>

  <!-- a convention instead of per-project settings: *.Tests projects never ship -->
  <PropertyGroup Condition="$(MSBuildProjectName.EndsWith('.Tests'))">
    <IsPackable>false</IsPackable>
  </PropertyGroup>
</Project>
```

Line 3 is the whole lesson. Without that `Import` the repository root's file never applies here. After it, read the rest as policy for every module: one `VersionPrefix` so the repo ships one version (8.4); `AnalysisLevel` + `EnforceCodeStyleInBuild` so analyzers and style rules run in the

build rather than in an IDE (7.2); lock files everywhere, switched to locked mode only when the `CI` environment variable is set, which is how a version bump stays easy locally and impossible to forget in CI (5.3). The last group is a *convention*: any project whose name ends `.Tests` is not packable, so nobody has to remember `IsPackable` per project.

4.3 · What the trap actually looks like

Terminal — a deliberately broken build — captured on the build machine by measure_build.py

```
# MotorMono/Directory.Build.props with its parent Import deleted
$ dotnet build src/L07.Core
error NETSDK1013: The TargetFramework value '' was not recognized. It may
be misspelled. If not, then the TargetFrameworkIdentifier and/or
TargetFrameworkVersion properties must be specified explicitly.
# exit code 1
```

The error names the symptom, not the cause. `TargetFramework` is empty because the only file that sets it — the repository root's `Directory.Build.props` — is no longer imported, and nothing in the message mentions imports. This is the diagnosis loop to learn: when a property is missing or wrong, run `dotnet msbuild <project> -getProperty:TargetFramework` to see its evaluated value, then `dotnet msbuild <project> -pp:pp.xml` and search the expanded document for the file you expected to set it (Customize the build by folder). **• Measured** by deleting the import in a temporary copy of the samples.

4.4 · The parent it chains to — and how one file serves C# and VB

MSBuild / XML — Directory.Build.props

```
<Project>
  <!--
    Applies to EVERY sample project in this repository: MSBuild walks up from each project folder
    and imports the first Directory.Build.props it finds. Lesson 07 explains this file line by line.
  -->
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Deterministic>true</Deterministic>
    <!-- bin/ and obj/ go to one place instead of beside every project -->
    <UseArtifactsOutput>true</UseArtifactsOutput>
    <ArtifactsPath>$(MSBuildThisFileDirectory).build\artifacts</ArtifactsPath>
    <!-- a sample that warns is a sample that teaches the wrong habit -->
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <!-- ...but a newly published package advisory must not break an old lesson overnight -->
    <WarningsNotAsErrors>NU1901;NU1902;NU1903;NU1904</WarningsNotAsErrors>
  </PropertyGroup>

  <PropertyGroup Condition="'$(MSBuildProjectExtension)' == '.csproj'">
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <PropertyGroup Condition="'$(MSBuildProjectExtension)' == '.vbproj'">
    <OptionStrict>On</OptionStrict>
    <OptionExplicit>On</OptionExplicit>
    <OptionInfer>On</OptionInfer>
  </PropertyGroup>
</Project>
```

The repository root's file is the one 4.2 imports, and it is where the two languages part ways. The first group applies to every project; the two groups below it are gated on `MSBuildProjectExtension`, so a `.csproj` gets `Nullable` and `ImplicitUsings` while a `.vbproj` gets `Option Strict On` — settings that mean something to one compiler and are noise to the other. One file, two languages, no per-project copy. `WarningsNotAsErrors` keeps a newly published NuGet advisory from breaking an old lesson overnight (§5).

4.5 · The Directory.* family — what each file may decide

build view

1 Directory.Build.props — the defaults

tooling · • Trap

Properties for every project below it. Imported EARLY, by Microsoft.Common.props, before the project file — so a project can still override anything it sets.

GRADLE	The `subprojects { }` block of a root build script
FINDS	MSBuild walks up from the project folder and stops at the first one it finds
TRAP	Stopping means stopping: a nested file must import its parent by hand
IN REPO	MotorMono's sets the version, analyzer level and lock-file policy

2 Directory.Build.targets — the last word

tooling · • Different

Targets and late properties. Imported LATE, after the project file and after any .targets from NuGet packages, so it sees what every project has already set.

GRADLE	A convention plugin, or `allprojects { }` applied last
USE FOR	Items and targets built from what the project decided — not for defaults
IN REPO	Adds `InternalsVisibleTo` when a matching .Tests folder exists, and a `ListProjects` target
NOTE	A property set here cannot be overridden by an individual project

3 Directory.Packages.props — the versions

tooling · • Renamed

One list of package versions for the whole repository, imported by the SDK's NuGet.props.

GRADLE	`gradle/libs.versions.toml` plus a `platform`
RULE	Projects name packages; only this file may carry a version
PINNING	`CentralPackageTransitivePinningEnabled` makes the list win for transitive packages too
NEAREST	Same first-hit-wins walk-up as Directory.Build.props

4.5 · The Directory.* family (continued)

build view

4 Directory.Build.rsp — the switches

tooling · • Same idea

Default command-line arguments for builds started below this folder.

GRADLE	`gradle.properties` with `org.gradle.*` switches
APPLIES	Command-line builds only — Visual Studio ignores it (MSBuild response files)
IN REPO	Turns off node reuse and stamps `MotorMonoBuiltFrom=command-line`
WATCH	A switch here is invisible in the IDE, so behaviour can differ per driver

5 Directory.Solution.props / .targets

tooling · • Different

Hooks for the solution build itself, not for the projects inside it.

RUNS WHEN	MSBuild builds a .slnx or .sln — never inside Visual Studio
SCOPE	Nothing it sets reaches the individual project builds
USE FOR	One-per-repository steps such as a report after every project is built
IN REPO	Not used — MotorMono keeps repo-wide steps in CI instead

5 · Dependencies — one version list for the repository

Central package management moves every version out of every project file into one `Directory.Packages.props`, and then forbids the old way. Set `ManagePackageVersionsCentrally`, list `PackageVersion` items, and a `PackageReference` that still carries a `Version` attribute is an error, not a warning (Central Package Management). That last part is the difference from a Gradle version catalog, which is a convenience you can bypass one module at a time.

The switch that matters is transitive pinning. A Gradle version catalog declares the versions you *ask* for — the docs are explicit that Gradle *"may still select different versions due to dependency graph conflicts or constraints applied through platforms"* (Gradle version catalogs). `CentralPackageTransitivePinningEnabled` is the `platform`/BOM half: a version in your list wins even for a package no project references directly. It has a library-author edge, and Central Package Management states it:

when you pack, a pinned transitive is promoted to an explicit dependency of your package. In an application repository that is what you want; in a published library it changes your public dependency set.

Two .NET 10 defaults change what restore does, in your favour. Auditing is now on for transitive packages, not just direct ones, whenever a project targets `net10.0` or later (restore audits transitive packages), and package pruning removes framework-provided packages from the graph for those projects — Microsoft measured *70% fewer transitive vulnerability reports* and restores *up to 50%* faster (NuGet package pruning in .NET 10). Both are measured as on in this repository (`NuGetAuditMode = all`, `RestoreEnablePackagePruning = true`). Audit warnings deliberately do *not* fail this repository's build: the root file excludes NU1901–NU1904 from warnings-as-errors so a newly published advisory cannot break an old lesson overnight.

5.1 · The repository's one version list

MSBuild / XML — `samples/MotorMono/Directory.Packages.props`

```
<Project>
  <PropertyGroup>
    <!-- Central Package Management: projects name packages, this file owns the versions -->
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
    <!-- a version listed here also wins when the package arrives transitively -->
    <CentralPackageTransitivePinningEnabled>true</CentralPackageTransitivePinningEnabled>
  </PropertyGroup>
  <ItemGroup>
    <PackageVersion Include="Microsoft.NET.Test.Sdk" Version="18.10.0" />
    <PackageVersion Include="xunit.v3.mtp-off" Version="4.0.1" />
    <PackageVersion Include="xunit.runner.visualstudio" Version="4.0.0" />
  </ItemGroup>
</Project>
```

Three packages, three versions, one file, whole repository. This is `gradle/libs.versions.toml` and a `platform` merged into one document — and unlike the catalog it is compulsory: with `ManagePackageVersionsCentrally` on, a project that names a version is rejected (5.3). The walk-up rule of §4 applies here too, so a nested `Directory.Packages.props` needs the same hand-written import. Lock files (`packages.lock.json`, one per project, committed) are what turn this list into a reproducible restore (locking dependencies). The three packages happen to be the test stack, which is the other half of *one list per repository*: these are the xUnit v3 packages lesson 10 (Testing, TDD & 100% Coverage) teaches — `xunit.v3.mtp-off` keeps them on VSTest — and that lesson owns the choice, including why mixing test platforms in one solution is not supported. Pick the framework before you copy a version list; it decides what the project in 5.2 looks like.

5.2 · The consumer side — references without versions

MSBuild / XML — `samples/MotorMono/tests/L07.Pricing.Tests/L07.Pricing.Tests.csproj`

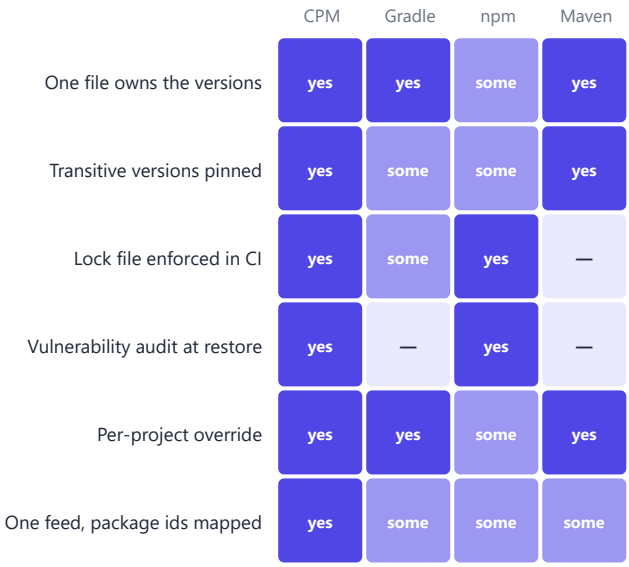
```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <!-- an xUnit v3 test project runs itself, so it is an executable, not a library -->
    <OutputType>Exe</OutputType>
  </PropertyGroup>
  <ItemGroup>
    <!-- no Version attributes: Directory.Packages.props owns every version -->
    <PackageReference Include="Microsoft.NET.Test.Sdk" />
    <PackageReference Include="xunit.v3.mtp-off" />
    <PackageReference Include="xunit.runner.visualstudio" PrivateAssets="all" />
  </ItemGroup>
  <ItemGroup>
    <Using Include="Xunit" />
    <ProjectReference Include="..\..\src\L07.Pricing\L07.Pricing.csproj" />
    <ProjectReference Include="..\..\src\L07.Pricing.Vb\L07.Pricing.Vb.vbproj" />
  </ItemGroup>
</Project>
```

Notice three conventions at once. The `PackageReference` items carry no version, because §5.1 owns them. The `Using` item adds a global `using Xunit;` for every file in the project — an MSBuild item that changes the C# compilation, not a source edit. And nothing here marks this as a test project that must not ship: `Directory.Build.props` already did that for every name ending `.Tests`, which is why `dotnet pack` produced 3 packages from 5 projects (8.4). The one property is not a build decision but a framework fact: an xUnit v3 test project runs itself, so it is an `Exe` rather than a library (lesson 10 (Testing, TDD & 100% Coverage)).

5.3 · Four ways a shared version list fails, and what restore says

What someone did	Code	Abridged message from the real run	The fix
Left `Version` on a `PackageReference` under CPM	• NU1008	The following PackageReference items cannot define a value for Version: xunit.v3.mtp-off. Projec…	Move the version to a `PackageVersion` item
Bumped a version and did not commit the lock file, with `CI=true`	• NU1004	The package reference xunit.v3.mtp-off version has changed from [4.0.1,) to [4.0.0,).The packa…	Restore locally, commit `packages.lock.json`
Added a second feed and no source mapping	• NU1507	Warning As Error: There are 2 package sources defined in your configuration. When using central…	Map every package id to one feed (package source mapping)
Referenced a package with a known vulnerability	• NU1901–NU1904	Audit warns at restore; on `net10.0` for transitive packages too (restore audits transitive packages)	Upgrade, or scope `WarningsNotAsErrors` deliberately

5.4 · What each ecosystem's shared version list actually guarantees



CPM = .NET 10 central package management · npm = npm workspaces · yes = built in and enforced · some = possible with a second mechanism or a plugin · — = not in the box · a rubric, not a benchmark

Read the second row first. Gradle scores *some* because the catalog declares and a `platform` enforces — two mechanisms (Gradle version catalogs) — where CPM plus `CentralPackageTransitivePinningEnabled` is one. Row 4 is the .NET 10 change: audit is in `dotnet restore` itself, at `all` by default for `net10.0` (restore audits transitive packages), where the JVM reaches for a plugin. Row 6 is the supply-chain one: `packageSourceMapping` binds each id to one feed, which is the answer to dependency confusion when you add a private feed (§8). • Estimate — the author's rubric of built-in behaviour, not a measurement.

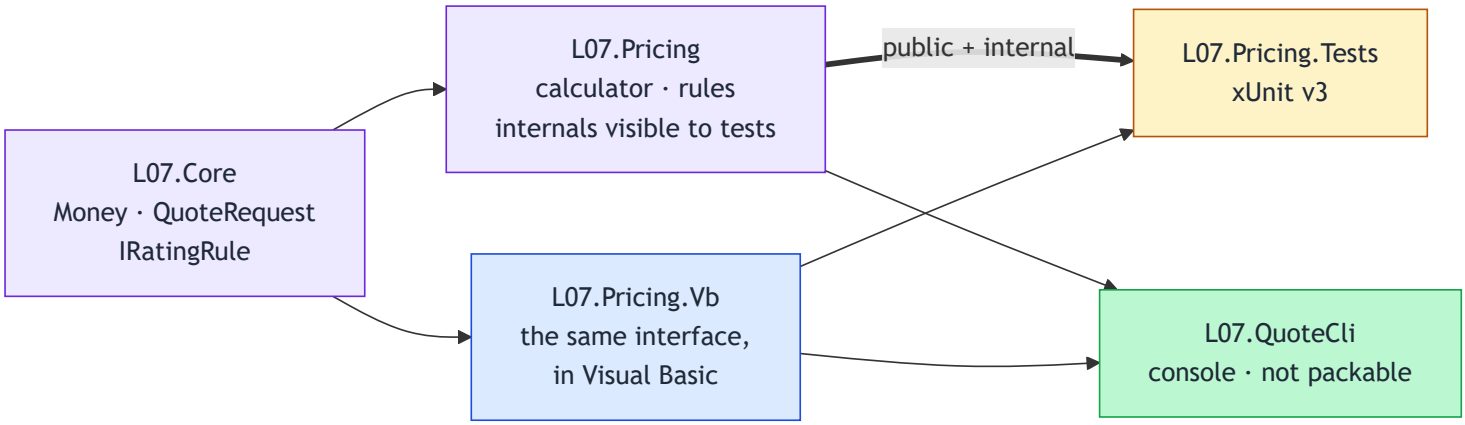
6 · The build graph — what recompiles when you change one line

The build order is not in the solution; it is the transitive closure of `ProjectReference` items. During execution, `ResolveProjectReferences` calls MSBuild on each referenced project and pauses until it returns, so the graph is walked depth-first as a tree of builds (How MSBuild builds projects). `L07.QuoteCli` in the samples never references `L07.Core`, yet compiles against it: a project reference is transitive, which is the opposite of Gradle's `implementation` and the same as its `api`.

What makes the graph cheap is the reference assembly. Every project also emits a metadata-only copy of itself, and Microsoft's description of why is exact: *"The reference assembly only changes when its public API is affected. So, using the reference assembly as an input file instead of the implementation assembly allows skipping the build of the dependent project in some cases"* (reference assemblies). Measured on this repository: a method body edit in `L07.Core` recompiles 1 project; changing a public signature recompiles all 5.

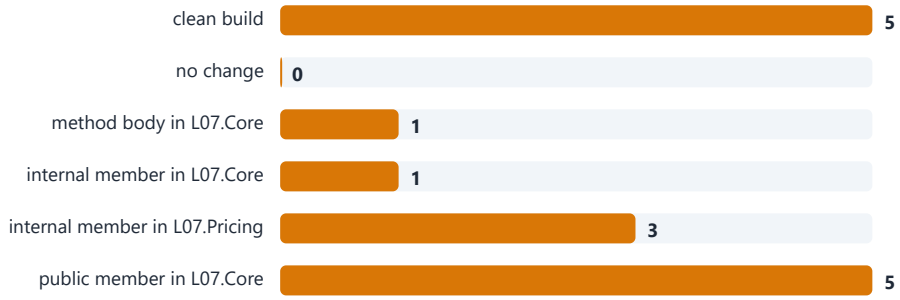
And then there is the case that breaks the rule. An internal member added to `L07.Core` recompiles 1 project — but the same edit in `L07.Pricing` recompiles 3. The reason is one line of `Directory.Build.targets`: `L07.Pricing` has a test project, so it gets `InternalsVisibleTo`, and *"If there are no `InternalsVisibleTo` attributes, internal function members are also removed"* from the reference assembly (reference assemblies · the Roslyn refout notes). Granting test access to internals is not free: it puts your internals in the public contract the build compares.

6.1 · The MotorMono build graph, and what each edge carries



violet = C# library · blue = Visual Basic library · green = console app · amber = tests · every thin edge is a project reference that carries the public API only, so an internal change stops there; the one thick edge also carries internals, because InternalsVisibleTo keeps them in the reference assembly

6.2 · Projects the compiler re-ran, by kind of edit



projects whose CoreCompile target actually invoked Csc or Vbc, counted from a detailed MSBuild log after each edit, on a temporary copy of the samples

The two middle bars are the same edit in two projects, and they differ. An internal member added to `L07.Core` costs 1 compile; the same member in `L07.Pricing` costs 3, because that project's reference assembly carries its internals for the test project. The ceiling is a public signature change: 5 of 5. Scale it mentally to a 120-module repository and the design rule writes itself — keep the widely referenced projects small and their public surface stable, and do not hand out `InternalsVisibleTo` to a project everything depends on. • **Measured**.

6.3 · internal in Kotlin and in C# — the same word, a different boundary

Kotlin — for comparison Kotlin — internal is per module, tests in…

```kotlin
// a Gradle module is one compilation unit, and
// src/main + src/test are the SAME module
internal object BaseRates {
    // share of the sum insured, illustrative
    fun forClass(c: CoverageClass): BigDecimal =
        when (c) {
            CLASS_1 -> "0.021".toBigDecimal()
            CLASS_3 -> "0.004".toBigDecimal()
            else -> throw IllegalArgumentException()
        }
}

// nothing to declare anywhere: the test source
// set already sees main's internal members
class BaseRatesTest {
    @Test fun class1() = assertEquals(
        "0.021".toBigDecimal(),
        BaseRates.forClass(CLASS_1))
}
```

C# samples/MotorMono/src/L07.Pricing/Internals.cs · #region int…

```csharp
// internal = visible inside the L07.Pricing assembly only.
// L07.Pricing.Tests is another assembly: it sees these
// types because Directory.Build.targets adds
// [InternalsVisibleTo("L07.Pricing.Tests")] at build time.
internal static class BaseRates
{
    // share of the sum insured per year: the tariff
    // the whole track prices with, illustrative only
    internal static decimal For(CoverageClass coverage) =>
        coverage switch
        {
            CoverageClass.Class1 => 0.021m,
            CoverageClass.Class2Plus => 0.012m,
            CoverageClass.Class3Plus => 0.009m,
            CoverageClass.Class3 => 0.004m,
            _ => throw new ArgumentOutOfRangeException(
                nameof(coverage)),
        };
}
```

Kotlin's module is a Gradle source set, and its test source set is inside it; .NET's module is the assembly, and the test project is a different one. Kotlin grants the test access by definition — the language reference lists *"A Gradle source set (with the exception that the `test` source set can access the internal declarations of `main`)"* (Kotlin visibility modifiers). In .NET nothing is visible across an assembly until `InternalsVisibleTo` says so, which is why this repository re-creates the Gradle convention with the glob condition in 3.4, and why the test in the samples can call `BaseRates.For` at all. The cost is measured in 6.2 — and the `ArgumentOutOfRangeException` arm exists because the C# compiler, unlike Kotlin's `when` over an enum, does not treat the switch as exhaustive.

6.4 · One build graph, two languages, one interface

C# samples/MotorMono/src/L07.Pricing/NoClaimBonus.cs · #region …

```csharp
/// <summary>The C# port of the shared no-claim
/// ladder: 20-50% off, none after a claim.</summary>
public sealed class NoClaimBonus : IRatingRule
{
    public string Name => "no-claim bonus";

    public decimal Factor(QuoteRequest request)
    {
        ArgumentNullException.ThrowIfNull(request);
        if (request.ClaimsLast5Years > 0)
        {
            return 0m;
        }

        return request.ClaimFreeYears switch
        {
            <= 0 => 0m,
            1 => -0.20m,
            2 => -0.25m,
            3 => -0.30m,
            4 => -0.40m,
            _ => -0.50m,
        };
    }
}
```

Visual Basic samples/MotorMono/src/L07.Pricing.Vb/NoClaimBonusVb.…

```vb
''' <summary>The original rule, still in Visual Basic,
''' the same ladder, in the same build graph.</summary>
Public NotInheritable Class NoClaimBonusVb
    Implements IRatingRule

    Public ReadOnly Property Name As String _
        Implements IRatingRule.Name
        Get
            Return "no-claim bonus"
        End Get
    End Property

    Public Function Factor(request As QuoteRequest) _
        As Decimal Implements IRatingRule.Factor
        ArgumentNullException.ThrowIfNull(request)
        If request.ClaimsLast5Years > 0 Then Return 0D

        Select Case request.ClaimFreeYears
            Case Is <= 0 : Return 0D
            Case 1 : Return -0.2D
            Case 2 : Return -0.25D
            Case 3 : Return -0.3D
            Case 4 : Return -0.4D
            Case Else : Return -0.5D
        End Select
    End Function
End Class
```

A mixed-language mono-repo needs no bridge: both projects implement the same `IRatingRule` from `L07.Core` and both are referenced by the same console app and the same test project. The build differences are real but small: the VB project needs `Implements IRatingRule.Name` on each member where C# infers it, and the repository root's `Directory.Build.props` has to split its settings by project extension so C# gets `Nullable` and VB gets `Option Strict On` (4.4). For a migration this is the whole strategy in one screen — port a rule to C#, keep both in the graph, and let the parity test in `L07.Pricing.Tests` prove they agree before the VB one is deleted (lesson 06 (VB.NET for Legacy Estates)). Both panels price with the same illustrative tariff as every other lesson that quotes this example — the same base rates, the same loadings and the same no-claim ladder, 20, 25, 30, 40 then 50% off for claim-free years — which is what makes a parity test between two languages worth writing. Three or more claims in five years is the one input that is not a loading: the tariff declines the quote, and a build lesson needs the smallest form of that outcome, so `L07.Core` defines a `QuoteDeclinedException` and lesson 02 (C# Language Essentials) owns exceptions properly.

7 · Quality gates that live inside the build

In .NET the linter is the compiler. Roslyn analyzers ship inside the SDK and run as part of every build, so there is no ktlint to add, no ESLint config to maintain and no separate lint task in CI. `AnalysisMode` — or the mode half of a compound `AnalysisLevel` such as `latest-recommended` — chooses how many rules are raised to warnings, `.editorconfig` overrides any single rule's severity, and `TreatWarningsAsErrors` decides whether a warning stops the build (code analysis in .NET). A SonarQube-style server pass becomes a property in a shared file.

The counts are worth knowing before you pick a mode. Counted in the SDK's own configuration files, `minimum` raises 92 CA rules to warning, `recommended` raises 145 and `all` raises 280. MotorMono sets `AnalysisLevel` to `latest-recommended`, the `recommended` mode, and the repository root turns warnings into errors, which is why no sample in this track can teach a habit the compiler dislikes. On an existing estate, go the other way: start at `minimum`, fix, raise.

Formatting is the one gate that is still a separate command, and it has a portability trap. `dotnet format --verify-no-changes` fails the build when a file is not formatted (dotnet format). Measured on this repository, on Windows: remove `end_of_line = lf` from `.editorconfig` and the same command exits 2, asking to replace characters with CRLF. On Linux, where the formatter's default line ending is LF, the same files pass — that half is reasoning, not a measurement. The pin only helps if the files on disk really are LF, which is the job of the sample's two-line `.gitattributes`; together they keep a green pipeline from meeting a red desk, or the reverse.

7.1 · The whole style configuration

Text samples/MotorMono/.editorconfig

```
# Style and analyzer rules for every module. The build enforces them: Directory.Build.props sets
# EnforceCodeStyleInBuild, and the repository root sets TreatWarningsAsErrors.
root = true

[*]
```

```
# pin line endings, or dotnet format --verify-no-changes can disagree between Linux CI and Windows
end_of_line = lf

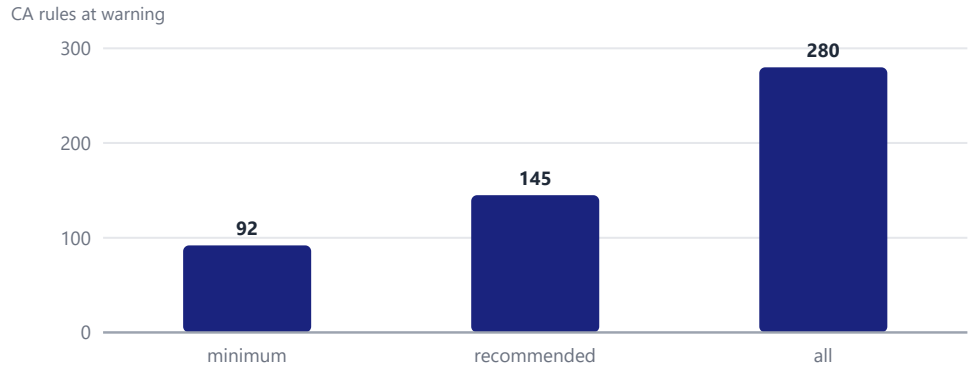
[*.{cs,vb}]
indent_style = space
indent_size = 4

[*.cs]
# a block-scoped namespace is a build error, not an IDE hint (.NET 9+ honours ":warning" on build)
csharp_style_namespace_declarations = file_scoped:warning

# tests read as sentences: allow underscores in test method names
[tests/**/*.cs]
dotnet_diagnostic.CA1707.severity = none
```

Four decisions, and each one is load-bearing. `end_of_line` pins line endings so `dotnet format` gives the same answer on every OS (7.3) — but only if the files on disk are LF, and that is the other half: `.gitattributes` holds `* text=auto eol=lf`. Without it, Git for Windows' default `core.autocrlf=true` checks CRLF files out and the same rule fails there instead (reasoning, not a measurement). The `csharp_style_namespace_declarations` rule is set to `:warning`, not `:suggestion`, so with `EnforceCodeStyleInBuild` and warnings-as-errors a block-scoped namespace is a build error — measured as `IDE0161` (IDE0161). The last section is the pattern to copy: relax a rule for a folder rather than for the repository, here letting test method names read as sentences with underscores. This file is also the only place a Gradle user needs no translation — it is the same `.editorconfig` your IDE already reads.

7.2 · How many code-analysis rules each analysis mode turns on



counted in the SDK's own analysislevel_10_<mode>.globalconfig files on the build machine, SDK 10.0.401

The jump from `recommended` to `all` is 135 more rules, and on an existing codebase most of them fire at once. This is why `AnalysisLevel` belongs in a shared file: one edit changes the gate for every module, and one project can still opt out locally. MotorMono runs `latest-recommended`, so 145 rules are warnings and — because the root file treats warnings as errors — any one of them can stop the build. • **Measured**; the rule sets move with every SDK, so re-count rather than quote this.

7.3 · Every gate in this repository, and what you knew it as

Gate	Switched on in	Fails the build when	You knew it as
Compiler warnings	repo root: `TreatWarningsAsErrors`	any C# or VB warning is raised	`-Werror` · `tsc` in strict mode
Code analysis (CA)	MotorMono: `AnalysisLevel` = `latest-recommended`	one of 145 rules fires	SonarQube rules, but in-build
Code style (IDE)	`.editorconfig` + `EnforceCodeStyleInBuild`	a rule set to `:warning` is violated — `IDE0161`	ktlint · ESLint rules
Formatting	`dotnet format --verify-no-changes` in CI, plus `.gitattributes eol=lf`	a file is unformatted or its line endings differ from `end_of_line`	`prettier --check` · `spotlessCheck`
Restore reproducibility	`RestorePackagesWithLockFile`, plus `RestoreLockedMode` when `CI` is set	a version moved and the lock file did not	`npm ci`
Vulnerabilities	`NuGetAuditMode` — `all` by default on `net10.0`	never here: the root excludes NU1901–NU1904 from errors	`npm audit` · Dependabot

8 · Packaging and CI — one workflow for the whole repository

Packing a mono-repo is one command over the solution, and the interesting decisions are about versions.
`dotnet pack MotorMono.slnx` produces one `.nupkg` per packable project — measured: 3 packages from 5 projects, because

test and executable projects opt out. Each project reference becomes a package dependency at the same version, so the measured `L07.Pricing` package depends on `L07.Core 1.4.0-ci.42`. Publish one module of a set and you have published a package whose dependency does not exist on the feed yet.

One property sets the version for every module. `VersionPrefix` lives in `Directory.Build.props`; CI appends a suffix on ordinary builds (`1.4.0-ci.42` was produced with `-p:VersionSuffix=ci.42`) and passes `-p:Version=` from the tag on a release. Order matters: a suffix is only applied when `Version` is not already set — *"If Version has a value and you pass a version-suffix, the value specified for the version-suffix is ignored"* (dotnet pack).

The workflow is ordinary GitHub Actions with three .NET-specific details. `setup-dotnet` installs the SDK from the repository's own `global.json`, so the runner uses the SDK this repository asks for — and that is a *range*, not one SDK: `10.0.401` or any later 10.0 feature band, because the file sets `rollForward` to `latestFeature`, *"the highest installed feature band and patch level that matches the requested major and minor"* (global.json and rollForward). The same page names the stricter setting and when to want it: *"When you use package lock files, set `rollForward` to `disable` so the SDK version and dependency graph stay in lockstep"* — which is exactly the combination §5 teaches, so a repository that commits lock files has one more decision than it looks like. The action's cache keys on the hash of those `packages.lock.json` files and fails outright if none exists (actions/setup-dotnet). GitHub sets `CI=true`, which is what flips this repository into locked restore mode. And `-bl` writes a binary log that is uploaded even when the build fails, so a red pipeline is debuggable without reproducing it.

8.1 · The build half of the workflow

YAML — samples/ci/build.yml · #region build

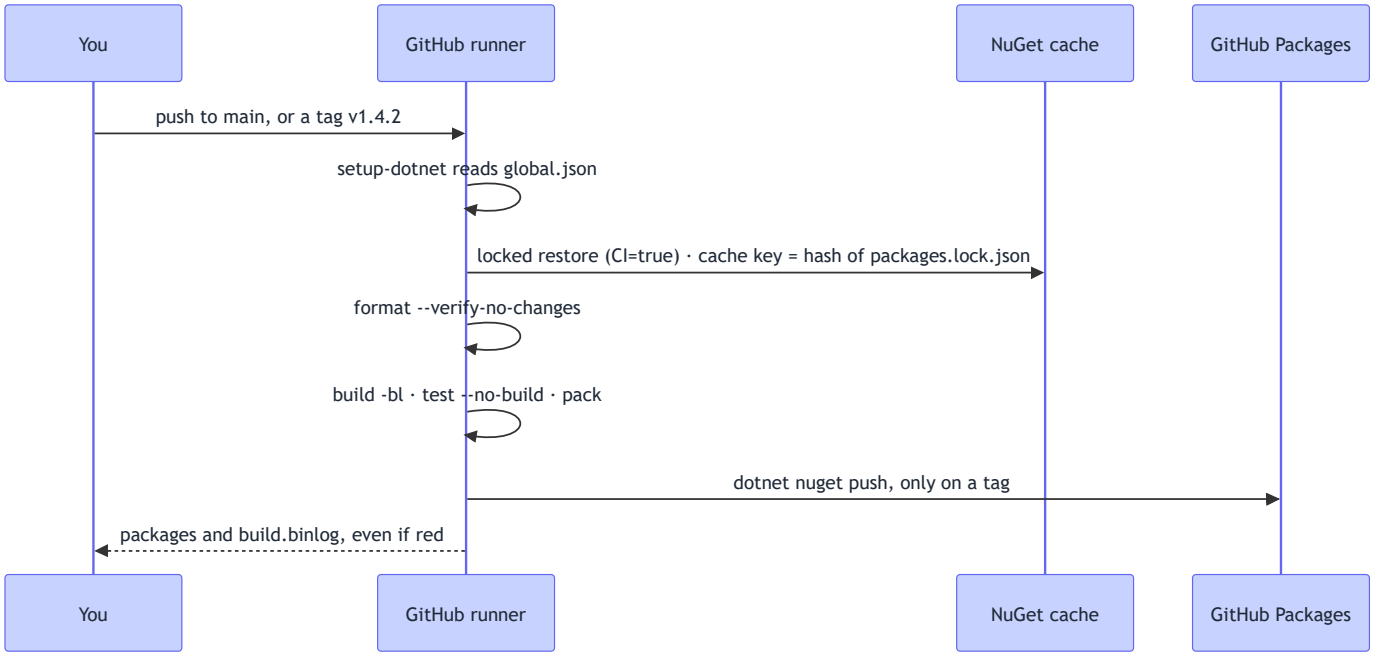
```yaml
- uses: actions/checkout@v7
- uses: actions/setup-dotnet@v6
  with:
    global-json-file: global.json # the SDK range the repo asks for (rollForward)
    cache: true # NuGet cache keyed on the hash of the lock files
    cache-dependency-path: "**/packages.lock.json"

# a tag v1.4.2 ships 1.4.2; every other build ships 1.4.0-ci.<run number>
- if: github.ref_type == 'tag'
  run: echo "VERSION_ARGS=-p:Version=${GITHUB_REF_NAME#v}" >> "$GITHUB_ENV"
- if: github.ref_type != 'tag'
  run: echo "VERSION_ARGS=-p:VersionSuffix=ci.${GITHUB_RUN_NUMBER}" >> "$GITHUB_ENV"

# GitHub sets CI=true, so Directory.Build.props switches restore to locked mode
- run: dotnet restore MotorMono.slnx
- run: dotnet format MotorMono.slnx --verify-no-changes --no-restore
- run: dotnet build MotorMono.slnx -c Release --no-restore $VERSION_ARGS -bl:build.binlog
- run: dotnet test MotorMono.slnx -c Release --no-build
- run: dotnet pack MotorMono.slnx -c Release --no-build $VERSION_ARGS -o packages
```

Five commands, in the only order that works. Restore first so every later step can pass `--no-restore`; format before build, because a formatting failure should be reported in seconds; build once and test with `--no-build` so the tested binaries are the built ones; pack with the same version arguments as the build. The two `if: github.ref_type` steps are the versioning rule from the story, written as environment variables rather than duplicated command lines. Note what is *not* here: no per-project steps, no matrix over modules — the solution file is the unit of work.

8.2 · One push through the pipeline



the runner reads global.json for the SDK range it may use, the lock files for the cache key, and the CI environment variable for locked restore; only a tag reaches the feed

8.3 · The publish half, and the secret that is not a secret

YAML `samples/ci/build.yml · #region publish`

```yaml
- uses: actions/upload-artifact@v7
  if: always() # keep the binary log of a failed build too
  with:
    name: packages-and-binlog
    path: |
      MotorMono/packages/*.nupkg
      MotorMono/build.binlog
- if: github.ref_type == 'tag'
  run: >
    dotnet nuget push "packages/*.nupkg" --api-key "${{ secrets.GITHUB_TOKEN }}"
    --source "https://nuget.pkg.github.com/${{ github.repository_owner }}/index.json"
```

`GITHUB_TOKEN` is enough to push to the repository owner's feed — the workflow's `permissions: packages: write` is what grants it, and the source URL is scoped to the owner, `nuget.pkg.github.com/<owner>/index.json` (the GitHub Packages NuGet registry). On AWS the same step becomes `aws codeartifact login --tool dotnet` before the push, which writes a token valid for at most 12 hours into the NuGet configuration (CodeArtifact with the dotnet CLI). Either way, add the feed to `nuget.config` *and* to `packageSourceMapping`: an unmapped second feed is the NU1507 row of 5.3 and, in a repository with private package ids, a dependency-confusion risk (package source mapping). The `if: always()` on the artifact upload is what makes the binary log useful; treat that file as sensitive — it records the environment variables that influenced the build (the MSBuild binary log).

8.4 · The packaging decisions, and what MotorMono chose

Decision	MotorMono	The alternative	Why
Where the version lives	`VersionPrefix` in `Directory.Build.props` — one version per repo	A version per project, or a git-height tool	One number to reason about; independent versions need independent release notes
CI versions	`-p:VersionSuffix=ci.N` → `1.4.0-ci.42`, and `-p:Version=` from the tag	Only tagged builds produce packages	Every build is installable, and pre-release ordering is a NuGet rule, not a convention
What ships	3 of 5 projects; anything ending `.Tests` and the console app opt out	`IsPackable` written per project	A naming convention in a shared file cannot be forgotten on a new project
Package dependencies	`L07.Pricing` → `L07.Core 1.4.0-ci.42`, generated from the project reference	Merge several projects into one package	Publish the set together, or a consumer restores a version that is not on the feed

Decision	MotorMono	The alternative	Why
Feed	GitHub Packages, owner-scoped, `GITHUB_TOKEN`	AWS CodeArtifact, Azure Artifacts, nuget.org	Whatever you pick, map the ids (package source mapping)
Reproducibility	`Deterministic` at the root, `ContinuousIntegrationBuild` when `CI` is set	Default (local) builds	Normalises paths in the output so two builds of one commit match

9 · Hands-on — drive MotorMono, then take it apart

Build it first, then inspect it — the second half is the part you will use at work. The whole mini mono-repo is 5 projects and 306 lines of C# and Visual Basic, so every command below finishes in seconds, and the four inspection commands are exactly what you run on a repository you have never seen: what is my evaluated property, which files set it, what targets exist, and what did the build actually do.

Then break it on purpose. `python samples/tools/measure_build.py` re-runs every measurement in this lesson on a temporary copy — the incremental-build table, the expanded-project line counts, the packaging results and all six deliberate failures — and rewrites `samples/tools/measurements.json`, which is where the numbers in this PDF come from. Reading that script is the fastest way to learn the diagnostic flags.

9.1 · Commands

Terminal

```
# from the repository root: build, test and run every lesson-07 project
python 0-script/verify_samples.py --only 7

# from lesson-07-solutions-monorepo-build/samples/MotorMono
dotnet build MotorMono.slnx                    # the whole graph, in dependency order
dotnet build MotorMono.Pricing.slnf            # a filtered subset (and its references)
dotnet test  MotorMono.slnx                    # every test project in the solution
dotnet run --project src/L07.QuoteCli          # the console app, output in 9.2

# inspect instead of build
dotnet msbuild src/L07.Core -getProperty:TargetFramework      # one evaluated property
dotnet msbuild src/L07.Pricing -pp:pp.xml                     # every import, expanded
dotnet msbuild src/L07.Core -t:ListProjects -tl:off           # one target, by name
dotnet build MotorMono.slnx -bl:build.binlog                  # a replayable recording

# re-measure everything this PDF prints (works on a temporary copy)
python lesson-07-solutions-monorepo-build/samples/tools/measure_build.py
```

The four inspection commands are the ones to memorise. `-getProperty:` answers *what is the value* without a build; `-pp:` answers *which file set it*; `-t:` runs one target, so repository tooling never slows an ordinary build; `-bl:` records the build for later (the MSBuild binary log). `-tl:off` turns off the terminal logger's live display when you are capturing output.

9.2 · What you should see

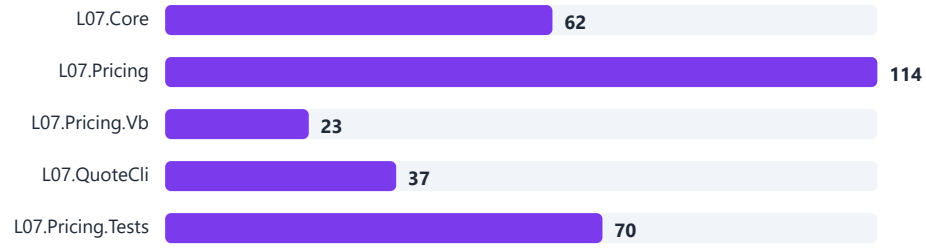
Output — dotnet run --project src/L07.QuoteCli captured on the build machine

```
MotorMono CLI 1.4.0  commit 13e8067
  projects   5
  built via  command-line

Class1 · sum insured 550,000.00 THB · driver age 23
  base premium                      11,550.00 THB
  young driver     L07.Pricing       2,310.00 THB
  no-claim bonus   L07.Pricing.Vb   -5,544.00 THB
  net premium                        8,316.00 THB
  stamp duty 0.4%                       33.26 THB
  VAT 7%                               584.45 THB
  total                              8,933.71 THB
rates are illustrative, not a real tariff
```

Every line of the header came from the build, not from the source. The version is `VersionPrefix`; the commit is stamped into `AssemblyInformationalVersion` because the SDK has shipped Source Link since .NET 8 (Source Link is in the SDK), so yours will differ from the one above; `projects 5` and `built via command-line` are the `AssemblyMetadata` items that `StampBuildFacts` wrote in 3.4 — run the app from an IDE and the last line reads `IDE`, because only the command line reads `Directory.Build.rsp`. The third column names the *module* each rating rule was compiled into: the mixed-language graph of 6.1 printing itself.

9.3 · Lines of code per module



non-blank, non-comment lines of C# and Visual Basic, measured from the sample files when this PDF was built

> **306 lines of code, and everything in this lesson is about the 5-line project files and the 74 lines of `Directory.*` around them.** That ratio is the honest reason build engineering is a discipline in a mono-repo: the shared files are small, they apply everywhere, and they are the only code in the repository with no tests. • **Measured** — the counts change as the samples change, and the point does not.

9.4 · What carries over from Gradle, what to learn, what to unlearn

Carries over unchanged

- Convention over per-module configuration
- One version source for the whole repository
- Lock files committed, enforced only in CI
- Quality gates inside the build, not beside it
- Keeping widely referenced modules small

Learn fresh

- Evaluation before execution, and the six ordered passes
- Nearest-file import, and chaining to the parent by hand
- Reference assemblies, and what they let the build skip
- `InternalsVisibleTo` as a build-cost decision
- `-pp`, `-getProperty`, `-t:`, `-bl`

Unlearn

- "The solution file configures the build" — it is an index
- "A solution filter isolates a subgraph" — it builds references too
- "An internal change is always cheap"
- "The project file tells me how this module builds"
- "A version catalog and central package management are the same thing"

⚠ Traps

1. **Only the nearest `Directory.Build.props` is imported.** Add one in a subfolder and everything the root file set stops applying below it, with no warning — the samples fail with `NETSDK1013` when the hand-written `Import` is removed (4.3). The same first-hit-wins rule applies to `Directory.Packages.props` and `Directory.Build.targets` (Customize the build by folder · Central Package Management).
2. **Granting test access to internals makes internal changes expensive.** `InternalsVisibleTo` keeps internal members in the reference assembly, so an internal edit in `L07.Pricing` recompiles 3 projects where the same edit in `L07.Core` recompiles 1 (6.2). Prefer testing through the public surface on the projects everything depends on (reference assemblies).
3. **A solution filter is not a build boundary.** `MotorMono.Pricing.slnf` lists 3 projects and 4 are built, because MSBuild follows references out of the filter (Solution filters in MSBuild). Use it to make an IDE open faster, not to prove a subsystem builds alone.
4. **A version catalog is not central package management.** Gradle's catalog declares the versions you request and a `platform` enforces them (Gradle version catalogs); CPM with `CentralPackageTransitivePinningEnabled` does both — and when you pack, a pinned transitive becomes an explicit dependency of your package, which quietly widens a published library's contract (Central Package Management).
5. **Pin `end_of_line`, make the disk agree, and treat `build.binlog` as sensitive.** Without `end_of_line = lf`, `dotnet format --verify-no-changes` exits 2 on Windows over LF files (measured; the Linux pass is reasoning). With it but without `.gitattributes eol=lf`, a CRLF checkout fails the rule the other way. And a binary log records the environment variables that influenced the build, so an uploaded artifact can leak more than you meant — do not set `MSBUILDLOGALLENVIRONMENTVARIABLES` in CI (the MSBuild binary log).

✓ Checkpoint — you are ready for lesson 08 when

1. `python 0-script/verify_samples.py --only 7` reports 5/5 passed, and `python .../tools/measure_build.py` reproduces the 6 incremental-build rows on your machine.
2. You can name every `Directory.*` file in the samples, say when it is imported and give one thing that belongs in it and one that does not.

3. Given an edit to a project in a mono-repo, you can predict which projects recompile — including the case where an internal change behaves like a public one.
4. You can take a repository with per-project package versions and no lock files and write the three-file change that centralises the versions, pins transitives and fails CI on drift.

🔍 Check yourself

1. A project sets a property in its `.csproj` and a `Directory.Build.targets` above it sets the same property. Which value does the build use, and why is the answer different for `Directory.Build.props`?
2. A 5-line project file evaluates to 17,412 lines. Which command shows you that document, and what is the first thing you search it for?
3. Why does adding an internal member to one project recompile one project, and adding an internal member to another recompile three?
4. `dotnet build MyFilter.slnf` builds a project the filter does not list. Is that a bug, and what would actually keep that project out of the build?
5. Under central package management, what happens if a `PackageReference` keeps its `Version` attribute — and what happens to your published package's dependencies if you pin a transitive version?
6. Your CI passes `-p:Version=1.4.2` and `-p:VersionSuffix=rc.1`. What version ships, and why?
7. Which two files make `dotnet build` on a developer machine and in Visual Studio behave differently, and how would you notice?

Lesson 07 in one line: a .NET mono-repo keeps its build above the projects — the solution file is only an index, `Directory.Build.props` applies to everything below it *until the next one shadows it*, `Directory.Packages.props` owns every version and pins transitives, and reference assemblies make the incremental cost of a change predictable: 1 project for an internal edit, 5 for a public one. Analyzers, `dotnet format` and a lock file are properties in a shared file, and one workflow packs the whole repository.
Next: lesson 08 (ASP.NET Core Web APIs) — the framework those projects exist to build.